

Guia de Avaliação Técnica: Infraestrutura de Sistemas Web

1. Visão Geral do Projeto e Objetivos

Este documento atua como a especificação técnica e critério de aceite para a implementação de infraestruturas de sistemas web. O arquiteto responsável deve optar por um dos dois caminhos tecnológicos: **Opção A (Docker/Orquestração Nativa)** ou **Opção B (AWS/Nuvem Gerenciada)**.

A implementação será considerada não-conforme se falhar em demonstrar a aplicação prática dos pilares de **escalabilidade, segurança e persistência de dados**. Independentemente do caminho escolhido, a solução deve seguir padrões de arquitetura de produção, priorizando a resiliência e a automação conforme os fundamentos de Engenharia DevOps.

2. Opção A: Infraestrutura Baseada em Contêineres (Docker)

Esta opção foca na virtualização em nível de sistema operacional e na orquestração de clusters.

2.1 Construção de Imagens Otimizadas (Dockerfile)

O arquiteto deve demonstrar o domínio sobre o sistema de camadas e a eficiência do build. Os requisitos para o **Dockerfile** são:

- **Multi-stage Builds:** Uso obrigatório de múltiplos estágios para separação total do ambiente de build (compilação) e runtime (execução), garantindo imagens leves e seguras.
- **Layer Caching e Ordenação:** As instruções devem ser ordenadas para otimizar o cache (Aula 003). Camadas menos mutáveis (instalação de dependências) devem preceder a cópia do código-fonte (**COPY . .**).
- **.dockerignore:** Implementação rigorosa para evitar o envio de lixo (logs, **node_modules**, pastas de IDE) ao daemon, reduzindo o contexto de build.
- **Templates Imutáveis:** A imagem deve ser tratada como um artefato imutável (Aula 002). Alterações em tempo de execução no sistema de arquivos do contêiner são estritamente proibidas.

2.2 Definição e Orquestração (Compose vs. Stack)

A arquitetura deve ser definida via **docker-compose.yml**, porém com visão de produção:

- **Estrutura Multicamadas:** Declaração obrigatória de no mínimo três serviços: App, DB (Banco de Dados) e Cache (Redis/Memcached).
- **Do Compose ao Cluster (Swarm):** Embora o Compose seja usado para a definição declarativa, o arquiteto deve diferenciar o ambiente single-host da orquestração multi-host. Se a entrega focar em produção, deve-se utilizar o conceito de **Stack** para gerenciar o ciclo de vida das réplicas e garantir a alta disponibilidade (TA008).

2.3 Rede e Comunicação Interna (Ponte Inteligente)

A comunicação entre serviços deve seguir o conceito de **Custom Bridge Networks** (Ponte Inteligente - TA005):

- **DNS Interno:** O uso de IPs estáticos para comunicação inter-contêiner é proibido. A resolução deve ocorrer via nome do serviço (Service Discovery nativo).
- **Isolamento:** A rede personalizada deve garantir que apenas os serviços necessários se enxerguem, provendo uma camada inicial de segurança perimetral.

2.4 Persistência de Dados

Para mitigar a natureza efêmera dos contêineres, é obrigatório o uso de **Named Volumes**. Bind mounts devem ser evitados para dados persistentes em produção, garantindo que o estado do banco de dados sobreviva à reinicialização ou deleção do contêiner.

3. Opção B: Infraestrutura em Nuvem (AWS)

Esta opção foca em serviços gerenciados e infraestrutura como código, visando a redução do overhead operacional.

3.1 Arquitetura de Rede (VPC e Defesa em Profundidade)

A **Amazon VPC** deve ser desenhada sob o conceito de "Planta Baixa" de rede segura (TA015):

- **Public Subnets:** Apenas para recursos de borda (Load Balancers ou Web Servers expostos).
- **Private Subnets:** Obrigatórias para bancos de dados e camadas de aplicação. **A exposição de bancos de dados em subnets públicas resultará em reprovação automática neste critério.**
- **Segurança em Camadas:** O arquiteto deve aplicar **Security Groups** (Stateful - Firewall de instância) como controle primário e granular, reservando **Network ACLs** (Stateless - Firewall de subnet) para regras amplas de bloqueio.

3.2 Segurança e Identidade (IAM)

A aplicação do **Princípio do Menor Privilégio** é mandatória:

- **Proibição de Credenciais Fixas:** É terminantemente proibido o uso de *Access Keys* e *Secret Keys* embutidas no código ou em arquivos de configuração (TA012).

- **IAM Roles:** O acesso a outros serviços (como S3) deve ser feito via Roles anexadas diretamente às instâncias EC2 ou Tarefas ECS, utilizando credenciais temporárias.

3.3 Computação e Escalabilidade (ECS/Fargate)

Para orquestração gerenciada, o uso de **Amazon ECS** com launch type **Fargate** (Serverless) é preferível para focar na aplicação:

- **Task Definitions:** Devem atuar como o blueprint da aplicação.
- **Services:** Devem ser configurados para gerenciar o número desejado de réplicas, provendo autorrecuperação e integração com Load Balancers.

3.4 Persistência e Paradigmas de Armazenamento

O arquiteto deve escolher o serviço baseado no paradigma correto (AWS Storage):

- **Amazon S3:** Armazenamento de Objetos (assets estáticos e backups) com durabilidade de **11 noves** (99.999999999%).
- **Amazon RDS:** Banco de Dados Relacional gerenciado (MySQL ou PostgreSQL). É obrigatória a configuração de **Multi-AZ** para failover automático em produção e a ativação de backups automáticos diários.
- **Decisão Arquitetural:** O arquiteto deve preterir o uso de instâncias EC2 para bancos de dados (gerenciamento manual) em favor do RDS, e justificar o uso de S3/RDS sobre EFS (File Storage) para este cenário de infraestrutura web.

4. Automação e Ciclo de Vida (CI/CD)

A integração com o **Amazon ECR (Elastic Container Registry)** é o ponto central da automação (TA012). O fluxo deve seguir as fases:

1. **Build:** Geração da imagem otimizada.
2. **Image Generation:** Tagging da imagem com a versão correspondente.
3. **ECR Push:** Envio da imagem para o registro privado da AWS.
4. **Deployment:** Atualização do serviço (ECS ou Swarm) consumindo a nova imagem do ECR.

5. Matriz de Avaliação e Pesos

Critério	Peso	Descrição do Atendimento Esperado
Eficiência da Imagem	20%	Uso de Multi-stage, camadas ordenadas (cache) e <code>.dockerignore</code> .

Arquitetura de Rede	25%	VPC com Private Subnets (AWS) ou Custom Bridge com DNS (Docker). DB em subnet pública = 0%.
Persistência de Dados	20%	Uso de RDS Multi-AZ e S3 (11 novos) ou Named Volumes resilientes.
Segurança e IAM	20%	IAM Roles sem chaves fixas (AWS) ou Isolamento de redes/usuários (Docker).
Automação (CI/CD)	15%	Pipeline integrado ao Amazon ECR com evidência de push/deploy.

6. Evidências Obrigatórias para Entrega

A defesa técnica exige a apresentação dos seguintes itens:

1. **Código-Fonte:** Arquivos `Dockerfile`, `docker-compose.yml` (ou manifestos de Infraestrutura).
2. **Prova de Domínio CLI:** O aluno deve apresentar a saída de comandos via terminal para provar que não dependeu exclusivamente da interface gráfica:
 - **Docker:** `docker service ps` ou `docker inspect <network_name>` para mostrar a resolução DNS.
 - **AWS:** `aws ec2 describe-instances` ou `aws ecs list-tasks` para validar recursos em execução.
3. **Logs do Pipeline:** Captura da saída do build da imagem e o sucesso do `docker push` para o ECR.
4. **PoC (Prova de Conceito):**
 - **Persistência:** Reinicializar o serviço e provar que os dados no banco persistem.
 - **Segurança:** Demonstração de que o banco de dados (na private subnet ou rede isolada) é inacessível diretamente pela internet, validando as regras de Security Group.
5. Um Vídeo narrado contendo a inicialização do ambiente e demonstração do sistema rodando.

7. Requisitos do README.md

Com base na estrutura do curso e nas melhores práticas de DevOps discutidas anteriormente, o **README.md** deve ser encarado pelos alunos como o **manual de operação e a planta arquitetural** do sistema.

O arquivo `README.md` é obrigatório e deve conter as seguintes seções, organizadas de forma clara e profissional:

1. Identificação do Projeto

- **Título:** Nome do sistema desenvolvido.
- **Descrição:** Breve explicação da funcionalidade do sistema.
- **Caminho Escolhido:** Identificação clara se o projeto segue a **Opção A (Docker/Orquestração Local)** ou a **Opção B (AWS/Cloud Infrastructure)**.

2. Pré-requisitos

- Lista de ferramentas necessárias instaladas no ambiente **WSL2** (ex: Docker Desktop, AWS CLI, kubectl).
- Configurações iniciais necessárias (ex: `aws configure` para credenciais ou login no Docker Hub/ECR).

3. Guia de Instalação e Execução ("How to Up")

- **Instruções passo a passo** de como subir a infraestrutura completa.
- Para **Docker (Opção A)**: Comandos para build e execução (ex: `docker-compose up -d --build`).
- Para **AWS (Opção B)**: Passos para deploy via CLI ou console (ex: criação do banco RDS, push para ECR e lançamento do EC2/ECS).

4. Detalhamento Técnico da Infraestrutura

- **Otimização de Imagens:** Breve explicação de como o **Dockerfile** foi otimizado (uso de imagens leves como *Alpine/Slim* e técnica de **Multi-stage build**).
- **Persistência:** Descrição da estratégia de dados usada (**Named Volumes** no Docker ou **Amazon RDS/S3** na AWS).
- **Rede e Comunicação:** Explicação de como os serviços se comunicam (uso de rede bridge customizada com **DNS Interno** no Docker ou estrutura de **VPC com subnets** na AWS).
- **Segurança:** Descrição das medidas aplicadas (variáveis de ambiente no arquivo `.env`, Roles do IAM ou Security Groups restritivos).

5. Gestão de Segredos e Configurações

- Instruções de como o avaliador deve configurar as variáveis de ambiente (ex: arquivo `.env.example`). **Aviso:** Reforce que nunca devem comitar senhas ou chaves reais no repositório.

6. Evidências de Funcionamento e Verificação

- Lista de comandos que o avaliador pode usar para validar o sistema (ex: `docker ps`, `docker-compose logs`, `aws sts get-caller-identity`).

- URL ou endereço IP de acesso à aplicação funcionando.

7. Troubleshooting e Limpeza

- Como diagnosticar problemas comuns e o comando para **destruir e limpar** os recursos após a avaliação para evitar custos desnecessários (ex: `docker-compose down -v` ou comandos de remoção na AWS).